ChessOTB.club — Game Recorder: Product Build Pack

Version: 2.0 — Launch-Ready | **Date:** March 2026 | **Status:** Pre-Build Review

---

Part 1 — Product Spec

1.1 What This Feature Actually Is

The Game Recorder is a **post-game analysis tool**, not a live engine assistant. Players finish their OTB game, then open ChessOTB.club on the same phone they used to record it. Within three minutes they have a full Stockfish analysis of every move — the same depth Chess.com gives for online games, but for the game they just played on a real board.

The core value proposition is a single sentence: **"Get Lichess-quality analysis for your OTB games, without a scoresheet."**

There are two input paths. The primary path for MVP is **manual PGN entry** — the player types or pastes their moves, or imports from a notation app. The secondary path, which ships as a beta feature, is **video-assisted reconstruction** — the player records the game with their phone camera and the system attempts to reconstruct the PGN from the video. Both paths converge at the same analysis screen.

1.2 MVP Scope — Hard Boundaries

The MVP is defined by what it does not do as much as what it does. The following table is the definitive scope boundary. Any feature not listed in the "In" column is out of scope for the initial launch.

In Scope	Out of Scope
Manual PGN entry with chess.js validation	Automated move reconstruction from video (beta only)
Video upload and storage (no processing)	Real-time board detection during recording
Stockfish analysis via Chess-API.com REST API	Self-hosted engine or higher-depth analysis
Per-move classification (best/good/inaccuracy/mistake/blunder)	Opening book integration or explorer
Accuracy percentage per player	ELO performance estimation
Interactive analysis board (react-chessboard)	Variation trees or alternative move exploration
Vertical eval bar (desktop) + horizontal (mobile)	Engine line display beyond best move
Color-coded move list with auto-scroll	PGN annotation editor
Key moments panel (top 5 eval swings)	Game sharing or public game links
My Games list on Profile page	Social features (comments, reactions)
Keyboard navigation (←→ Home End F)	Mobile touch swipe navigation
Board flip (White/Black perspective)	Piece animation speed settings
Video recording via MediaRecorder API	Video editing or trimming
Video playback in analysis (no sync)	Video-to-move timestamp synchronization (post-MVP)

The **video-to-board sync** feature is explicitly deferred. It requires a reliable timestamp mapping system that depends on the vision pipeline being accurate — which is not guaranteed in MVP. The analysis screen will show the video player and the board side-by-side, but they will not be linked. Users can scrub the video manually while navigating the board independently.

1.3 User Segments and Entry Points

Segment A — Club Tournament Player. Plays in ChessOTB.club events. Wants to review their round games the same evening. Entry point: the “Analyze” link in the main nav, or a future “Analyze This Game” button on the tournament matchup page.

Segment B — Casual OTB Player. Plays at a chess club, cafe, or home. Wants a game record without maintaining a scoresheet. Entry point: the “Analyze” nav link. May not be signed in — the feature requires authentication to save games.

Segment C — Coach or Serious Improver. Reviews multiple games per week. Wants accuracy trends over time. Entry point: the My Games section on the Profile page. This segment is the primary driver for the post-MVP “Personal Analytics” roadmap.

1.4 Mobile Recording Flow — Refined UX

The recording flow is the most physically constrained part of the product. The user is at a chess board, their phone is propped up, and they cannot interact with the screen during the game. Every screen in this flow must work with **one tap** and must be readable from 60 cm away.

Screen R1 — Record Entry (`/record`)

The entry screen has two modes depending on auth state.

Signed out: A single card with the feature headline, a brief description, and a "Sign In to Analyze" button. No other content. Do not show the PGN form to signed-out users — it creates false expectations.

Signed in, no games: The full entry screen with two options: "Enter PGN" (primary, prominent) and "Record with Camera" (secondary, with "Beta" badge). Below the options, a "How it works" section with three steps: Record → Analyze → Review.

Signed in, has games: The entry screen shows the "My Games" list first, with a floating "+" button to start a new analysis. This is the returning user state.

The "Enter PGN" path is the primary CTA. It should be visually dominant — a large card, not a small button.

Screen R2 — PGN Entry

The PGN entry form has five fields: White Player, Black Player, Result (segmented control: White Wins / Draw / Black Wins), Date (auto-filled today), and the PGN textarea. The textarea is the dominant element — it takes up 40% of the screen height.

Validation feedback is inline and immediate. As the user types, a status chip below the textarea shows: "Waiting…" (grey), "Valid — 32 moves" (green), or the specific error (red). The submit button is disabled until the PGN is valid.

Paste shortcut is a prominent button next to the textarea label. On iOS/Android, this triggers the clipboard read permission prompt. The button text is "Paste from Clipboard", not an icon.

Load Sample is a text link, not a button, placed below the textarea. It loads a well-known game (e.g., Kasparov vs Karpov, 1985 Game 16) for users who want to try the analysis before entering their own game.

Form submission creates a `RecordingSession` and a `ProcessedGame`, then immediately triggers engine analysis and navigates to the analysis page. The user sees the analysis page loading while Stockfish runs in the background.

Screen R3 — Camera Setup (`/record/camera`)

This screen is only shown when the user selects “Record with Camera”. It is a full-screen camera preview with a minimal overlay.

The overlay has four elements: (1) a dashed rectangle showing the ideal board framing area, (2) a status chip at the top (“Position the board inside the frame”), (3) a large “Start Recording” button at the bottom, and (4) a small “Enter PGN Instead” escape hatch in the top-right corner.

The status chip has three states: **“Frame the board”** (default, grey), **“Hold steady”** (amber, when the board is partially visible), and **“Ready to record”** (green, when the board fills the frame adequately). The “Start Recording” button is enabled in all states — the framing check is advisory, not a gate.

Critical UX decision: Do not block recording on board detection. If the user’s phone cannot detect the board (poor lighting, non-standard set, unusual angle), they should still be able to record. The video is stored regardless; the vision pipeline can attempt reconstruction later, and the user can always fall back to manual PGN entry.

Screen R4 — Recording In Progress

This is the most important screen to get right. The user sets their phone down and walks away. The screen must:

1. Stay awake (implement `navigator.wakeLock.request('screen')`)
2. Show elapsed time in a large, readable font (minimum 48px)
3. Show a red pulsing dot indicating active recording
4. Have a single large “Stop” button — minimum 64px touch target
5. Show battery level warning if below 20%

No other UI elements. No navigation. No distractions. The screen background should be near-black to conserve battery.

The recording uses `MediaRecorder` with `video/webm;codecs=vp9` on Android and `video/mp4` on iOS (Safari). The recorded blob is held in memory during recording and uploaded immediately after the user taps Stop.

Screen R5 — Processing Status (`/record/:sessionId/processing`)

A focused status screen with a four-step progress indicator:

1. **Uploading video** — shows upload percentage and file size
2. **Queued for analysis** — shows estimated wait time
3. **Running Stockfish** — shows moves analyzed / total moves
4. **Analysis complete** — transitions to analysis page

For the MVP, steps 1 and 3 are the only active steps (video upload + engine analysis). Step 2 is a brief transition state. The vision pipeline (board detection, move reconstruction) is a background process that runs asynchronously and does not block the user from viewing the analysis.

If the user entered PGN manually, this screen is skipped entirely — the app navigates directly to the analysis page with a “Analyzing…” indicator in the header.

1.5 Post-Game Analysis Screen — Refined Layout

The analysis screen is where users spend the most time. The layout must prioritize the board and the move list above everything else.

Mobile Layout (primary)

The mobile layout is a single scrollable column with a sticky header and a sticky bottom navigation strip.

Sticky header (56px): Back arrow, game title (“Kasparov vs Karpov · 1-0”), analysis progress indicator (if still running).

Board section (full-width square): The chessboard fills the full viewport width. Below the board, a horizontal eval bar (16px tall) spans the full width. Below the eval bar, a one-line move info strip shows the current move classification and best move alternative.

Move navigation strip (sticky bottom, 56px): Five buttons: ⟨⟨ ⟨ Flip ⟩ ⟩⟩. This strip is always visible, fixed to the bottom of the screen above the browser chrome. Keyboard shortcuts are not relevant on mobile — these buttons are the primary navigation.

Scrollable content below the fold:

- Move list (full-width, 2-column pair layout)
- Accuracy comparison card (White vs Black, side by side)
- Game info card (event, date, result, opening)
- Key moments list (top 5 eval swings, tappable to jump to position)
- Video player (if a recording exists, shown last)

The video player is below the fold on mobile. This is intentional. Most users will interact with the board and move list first. The video is a supplementary reference, not the primary interface.

Desktop Layout (two-column)

Left column (flexible, max 600px):

- Chessboard (square, fills column width)
- Horizontal eval bar below board
- Move info strip

- Navigation controls
- Video player (below controls, 16:9)

Right column (360px fixed):

- Move list (scrollable, max-height 400px)
- Accuracy comparison card
- Game info card
- Key moments list

The eval bar is horizontal on both mobile and desktop in this layout. The vertical sidebar eval bar (the current implementation) is visually elegant but takes up 32px of horizontal space that the board needs on smaller desktops. Replace it with a horizontal bar below the board on all breakpoints.

Analysis Loading States

The analysis page has three distinct loading states that must be handled explicitly:

State 1 — Game loaded, analysis pending: The board shows the starting position. The move list shows the moves but without color-coding. A banner reads "Stockfish is analyzing your game — this takes about 30 seconds." The navigation controls are functional — the user can step through the moves immediately.

State 2 — Analysis in progress (partial results): As moves are analyzed, the move list updates in real-time with color-coded classifications. The eval bar updates as the user navigates to analyzed positions. Unanalyzed positions show a grey eval bar.

State 3 — Analysis complete: The full analysis is available. The banner disappears. The accuracy panel and key moments panel appear.

This progressive loading approach means users can start reviewing their game immediately after submitting the PGN, without waiting for the full Stockfish analysis to complete.

1.6 Manual Correction Flow — Refined UX

The correction flow is only relevant when the vision pipeline has been run (video recording path). For the MVP, it is a secondary feature. The design must be clear about what is being corrected and why.

When It Appears

The correction flow appears after the processing status screen, only when the vision pipeline has flagged one or more moves as uncertain (confidence below 70%). If no corrections are needed, the app navigates directly to the analysis page.

Correction Screen Design

The correction screen is a focused, distraction-free interface. It shows:

Top section (40% of screen): The chessboard at the position just before the uncertain move. The last confirmed move is highlighted. A label reads “Move 18 — White to play.”

Middle section (30% of screen): The video frame at the corresponding timestamp, shown as a still image with a “Play ±5s” button to watch the surrounding video. A label reads “Check the video to confirm what happened.”

Bottom section (30% of screen): Two or three candidate move buttons, each showing the move in SAN notation and a small board thumbnail of the resulting position. A “None of these” button opens a mini-board for manual piece dragging.

Progress indicator: “3 of 7 corrections remaining” at the top of the screen.

Skip button: A small “Skip — use AI’s best guess” link below the candidate buttons. Skipped moves are marked with a ⚠ icon in the move list on the analysis page.

Correction Batching Logic

Corrections are presented in chronological order. Each correction is independent — the user can skip any of them. After all corrections are resolved (or skipped), the app triggers the final PGN generation and navigates to the analysis page.

Do not re-analyze the entire game after corrections. Only re-analyze the positions that were corrected. This keeps the correction-to-analysis transition fast (under 10 seconds for most games).

1.7 System Status Logic — Definitive State Machine

The `RecordingSession.status` field is the single source of truth for the entire feature’s UI state. The following table is the complete, authoritative state machine.

Status	Trigger	UI State	Next States
`draft`	Session created (before any input)	Entry screen, no progress	`pgn_submitted`, `recording`
`pgn_submitted`	PGN submitted via form	Processing screen (brief)	`analyzing`
`recording`	MediaRecorder started	Recording screen	`uploading`
`uploading`	MediaRecorder stopped, upload started	Processing screen — step 1	`upload_complete`, `upload_failed`
`upload_complete`	S3 upload finished	Processing screen — step 2	`vision_processing`, `analyzing`
`vision_processing`	Vision pipeline started	Processing screen — step 3	`needs_correction`, `pgn_ready`, `vision_failed`
`needs_correction`	Vision pipeline flagged uncertain moves	Correction screen	`pgn_ready`
`pgn_ready`	PGN confirmed (manual or vision)	Processing screen — step 4	`analyzing`
`analyzing`	Engine analysis started	Analysis page with progress banner	`complete`
`complete`	All moves analyzed	Analysis page, full results	—
`upload_failed`	S3 upload failed	Error screen	`uploading` (retry)
`vision_failed`	Vision pipeline failed entirely	Error screen with “Enter PGN” CTA	`pgn_submitted`

Polling strategy: The client polls `GET /api/recordings/:id` every 3 seconds when the session is in a transitional state (`uploading`, `vision_processing`, `analyzing`). Polling stops when the status reaches `complete`, `upload_failed`, or `vision_failed`. The polling interval backs off to 10 seconds after 2 minutes to reduce server load for long-running analyses.

Error recovery: Every error state must offer at least two recovery paths: retry the failed step, or fall back to manual PGN entry. Never leave the user on a dead-end error screen.

Part 2 — Prioritized Engineering Task List

Tasks are grouped by sprint and ordered by dependency. Each task has an estimated complexity (S = half day, M = 1 day, L = 2 days, XL = 3+ days).

Sprint 1 — Core Analysis Flow (Week 1)

The goal of Sprint 1 is a working end-to-end flow: user enters PGN → analysis runs → user sees results. No video, no recording.

#	Task	Complexity	Dependencies
1.1	Refactor `RecordingSession` status to the 12-state machine defined in § 1.7	S	—
1.2	Implement progressive analysis loading: game loads immediately, analysis updates in real-time	M	1.1
1.3	Replace vertical eval sidebar with horizontal eval bar below board on all breakpoints	S	—
1.4	Add sticky bottom navigation strip on mobile (⟪ ⟨ Flip ⟩ ⟫)	S	—
1.5	Implement touch swipe navigation on analysis board (swipe left = next move, right = previous)	M	1.4
1.6	Add move info strip below eval bar (classification chip + best move alternative)	S	—
1.7	Implement analysis polling with exponential backoff	S	1.1
1.8	Add “Analyzing…” banner to analysis page header during `analyzing` state	S	1.7
1.9	Fix `/record` entry screen: show My Games list for returning users	M	—
1.10	Add `GET /api/recordings` endpoint to list user’s sessions	S	—

Sprint 2 — Recording Flow (Week 2)

The goal of Sprint 2 is a working camera recording flow that uploads video to S3 and stores it against the session.

#	Task	Complexity	Dependencies
2.1	Build `/record/camera` page with live camera preview (MediaDevices API)	M	—
2.2	Implement board framing overlay (dashed rectangle, status chip)	S	2.1
2.3	Implement `navigator.wakeLock.request('screen')` for recording screen	S	—
2.4	Implement `MediaRecorder` with `video/webm` (Android) + `video/mp4` (iOS) fallback	M	2.1
2.5	Build recording in-progress screen: large timer, red dot, Stop button	S	2.4
2.6	Implement chunked video upload to S3 via `storagePut` after recording stops	L	2.4
2.7	Build `/record/:sessionId/processing` status screen with 4-step progress indicator	M	1.1
2.8	Add battery level warning (< 20%) to recording screen	S	2.5
2.9	Add `POST /api/recordings/:id/upload` endpoint (receives video, stores to S3, updates status)	M	—
2.10	Add video player to analysis page (below the fold on mobile, left column on desktop)	M	—

Sprint 3 — Correction Flow + Polish (Week 3)

The goal of Sprint 3 is the manual correction flow and overall UX polish.

#	Task	Complexity	Dependencies
3.1	Build `CorrectionCard` component: board + video still + candidate move buttons	L	—
3.2	Build correction batch flow: sequential cards, progress indicator, skip logic	M	3.1
3.3	Implement “None of these” fallback: mini-board with drag-to-move	L	3.1
3.4	Add ⚠ icon to skipped corrections in move list	S	3.2
3.5	Implement partial re-analysis after corrections (only corrected positions)	M	3.2
3.6	Add `POST /api/recordings/:id/corrections` endpoint	S	—
3.7	Add signed-out state to `/record` entry screen	S	—
3.8	Add “Enter PGN Instead” escape hatch to camera setup screen	S	—
3.9	Implement error recovery screens for `upload_failed` and `vision_failed`	M	1.1
3.10	Add My Games section to Profile page	M	1.10

Sprint 4 — Vision Pipeline Beta (Week 4)

The goal of Sprint 4 is a best-effort vision pipeline that attempts to reconstruct moves from uploaded video. This is a beta feature — it may fail, and the fallback to manual PGN entry must be seamless.

#	Task	Complexity	Dependencies
4.1	Implement server-side frame extraction from uploaded video (FFmpeg, 1 fps)	L	2.9
4.2	Integrate chess-snapshot-api or OpenAI Vision for FEN extraction from frames	XL	4.1
4.3	Implement move inference from FEN sequence (detect state changes, validate with chess.js)	L	4.2
4.4	Implement confidence scoring for inferred moves	M	4.3
4.5	Trigger correction flow for moves below confidence threshold	M	4.4, 3.2
4.6	Add “Beta” badge to camera recording option on entry screen	S	—
4.7	Add vision pipeline status updates to processing screen	S	4.1

Part 3 — Component Inventory

Every component required for the Game Recorder feature, with its location, props interface, and current status.

Page Components

Component	Route	Status	Notes
`GameRecorder`	`/record`	Built — needs refactor	Add My Games list, signed-out state, returning user flow
`CameraSetup`	`/record/camera`	Not built	New page — Sprint 2
`RecordingActive`	`/record/camera` (modal overlay)	Not built	Overlay on CameraSetup — Sprint 2
`ProcessingStatus`	`/record/:sessionId/processing`	Not built	New page — Sprint 2
`CorrectionFlow`	`/record/:sessionId/corrections`	Not built	New page — Sprint 3
`GameAnalysis`	`/game/:gameId/analysis`	Built — needs refinement	Progressive loading, mobile nav strip, eval bar position

Shared Components

Component	File	Props	Status
`EvalBar`	`components/EvalBar.tsx`	`evalCp: number`, `orientation: 'h' | 'v'`	Built in GameAnalysis.tsx — extract to shared
`MoveList`	`components/MoveList.tsx`	`analyses`, `currentIndex`, `onSelect`	Built in GameAnalysis.tsx — extract to shared
`AnalysisBoard`	`components/AnalysisBoard.tsx`	`fen`, `orientation`, `lastMove`, `bestMove`	Inline in GameAnalysis.tsx — extract to shared
`MoveNavStrip`	`components/MoveNavStrip.tsx`	`onFirst`, `onPrev`, `onFlip`, `onNext`, `onLast`, `canGoBack`, `canGoForward`	Not built — Sprint 1
`MoveInfoStrip`	`components/MoveInfoStrip.tsx`	`analysis: MoveAnalysis | null`	Not built — Sprint 1
`AccuracyPanel`	`components/AccuracyPanel.tsx`	`summary: AnalysisSummary`, `game: GameData`	Built in GameAnalysis.tsx — extract to shared
`KeyMomentsPanel`	`components/KeyMomentsPanel.tsx`	`moments: KeyMoment[]`, `onSelect`	Built in GameAnalysis.tsx — extract to shared
`GameInfoCard`	`components/GameInfoCard.tsx`	`game: GameData`	Built in GameAnalysis.tsx — extract to shared
`CameraPreview`	`components/CameraPreview.tsx`	`onStream`, `onError`	Not built — Sprint 2
`BoardFramingOverlay`	`components/BoardFramingOverlay.tsx`	`detectionState: 'idle' | 'partial' | 'ready'`	Not built — Sprint 2

Component	File	Props	Status
`RecordingTimer`	`components/RecordingTimer.tsx`	`startedAt: Date`, `onStop`	Not built — Sprint 2
`ProcessingSteps`	`components/ProcessingSteps.tsx`	`currentStep`, `steps: Step[]`	Not built — Sprint 2
`CorrectionCard`	`components/CorrectionCard.tsx`	`position`, `candidates`, `videoUrl`, `timestamp`, `onSelect`, `onSkip`	Not built — Sprint 3
`MyGamesList`	`components/MyGamesList.tsx`	`sessions: RecordingSession[]`	Not built — Sprint 1
`GameCard`	`components/GameCard.tsx`	`game: ProcessedGame`, `session: RecordingSession`	Not built — Sprint 1

Hooks

Hook	File	Purpose	Status
`useAnalysisNavigation`	`hooks/useAnalysisNavigation.ts`	Manages current move index, keyboard shortcuts, swipe gestures	Not built — extract from GameAnalysis
`useAnalysisPolling`	`hooks/useAnalysisPolling.ts`	Polls analysis endpoint with backoff, returns live data	Not built — Sprint 1
`useMediaRecorder`	`hooks/useMediaRecorder.ts`	Wraps MediaRecorder API, handles iOS/Android codec differences	Not built — Sprint 2
`useWakeLock`	`hooks/useWakeLock.ts`	Requests and releases screen wake lock	Not built — Sprint 2
`useBoardDetection`	`hooks/useBoardDetection.ts`	Runs lightweight board detection on camera frame	Not built — Sprint 4
`useSessionStatus`	`hooks/useSessionStatus.ts`	Polls session status, drives UI state machine	Not built — Sprint 1

API Endpoints

Method	Path	Purpose	Status
`POST`	`/api/recordings`	Create new session	Built
`GET`	`/api/recordings`	List user's sessions	Not built — Sprint 1
`GET`	`/api/recordings/:id`	Get session status + metadata	Built
`POST`	`/api/recordings/:id/pgn`	Submit manual PGN	Built
`POST`	`/api/recordings/:id/upload`	Upload video to S3	Not built — Sprint 2
`POST`	`/api/recordings/:id/analyze`	Trigger engine analysis	Built
`POST`	`/api/recordings/:id/corrections`	Submit move corrections	Not built — Sprint 3
`GET`	`/api/games/:id/analysis`	Get full analysis results	Built
`DELETE`	`/api/recordings/:id`	Delete session + game	Not built — Sprint 3

Part 4 — Launch-Ready MVP Checklist

This checklist defines the minimum bar for a public launch. Every item must be checked before the feature goes live.

Functional Requirements

- [ ] User can enter a PGN and receive a complete Stockfish analysis
- [ ] PGN validation provides clear, specific error messages (not just “invalid”)
- [ ] Analysis page loads immediately after PGN submission (no waiting screen)
- [ ] Analysis updates progressively as Stockfish processes each move
- [ ] User can navigate through all moves using on-screen buttons
- [ ] User can navigate through all moves using keyboard arrow keys (desktop)
- [ ] Board flips correctly between White and Black perspective
- [ ] Eval bar updates correctly for every move, including mate-in-N positions
- [ ] Move list auto-scrolls to keep current move visible
- [ ] Move classifications are color-coded correctly (green/yellow/orange/red)
- [ ] Accuracy percentages are calculated and displayed for both players
- [ ] Key moments panel shows the top 5 evaluation swings
- [ ] My Games list shows all of the user’s past analyzed games
- [ ] User can delete a game from My Games
- [ ] Signed-out users see a sign-in prompt, not a broken form

Mobile UX Requirements

- [ ] Analysis board is full-width on mobile (no horizontal scroll)
- [ ] Sticky bottom navigation strip is visible and functional on all mobile browsers
- [ ] Touch targets for navigation buttons are minimum 44px × 44px
- [ ] Swipe left/right navigates moves on mobile
- [ ] Eval bar is horizontal and below the board on mobile
- [ ] Move list is readable without horizontal scrolling
- [ ] PGN textarea is large enough to type comfortably on mobile
- [ ] “Paste from Clipboard” button works on iOS Safari and Android Chrome
- [ ] Form inputs do not cause layout shift when the keyboard appears

Performance Requirements

- [ ] Analysis page first contentful paint under 1.5 seconds on 4G
- [ ] Board position updates are smooth (no jank on move navigation)
- [ ] Stockfish analysis completes within 60 seconds for a 40-move game
- [ ] Polling does not cause visible UI flicker or layout shifts
- [ ] My Games list loads within 500ms for up to 50 games

Error Handling Requirements

- [ ] Network error during analysis submission shows a retry button
- [ ] Chess-API.com timeout (> 10 seconds) shows a “Taking longer than usual” message
- [ ] Invalid FEN from analysis API is handled gracefully (skip that move, continue)
- [ ] Session not found (deleted or expired) shows a clear error and link to start over
- [ ] Browser without MediaRecorder support shows “Camera recording not supported” with PGN fallback

Data Integrity Requirements

- [ ] PGN is stored verbatim in the database (no lossy transformation)
- [ ] Analysis results are idempotent — re-running analysis on the same PGN produces the same results
- [ ] Move analyses are associated with the correct game ID (no cross-contamination)
- [ ] User can only access their own games (no IDOR vulnerability)
- [ ] Session tokens are validated on every API request

Content and Copy Requirements

- [ ] Feature name is consistent across all screens: “Analyze” in nav, “Game Analysis” on page titles
- [ ] No placeholder text (“Lorem ipsum”, “TODO”, “Coming Soon” without a date) in any user-facing string
- [ ] Error messages are written in plain English, not technical codes
- [ ] Empty states have a clear call-to-action (not just “No games found”)
- [ ] The “Camera Recording — Beta” label is present and explains what “Beta” means on hover/tap

Testing Requirements

- [ ] Unit tests cover PGN validation edge cases (empty, invalid moves, headers, results)

- [ ] Unit tests cover eval bar percentage calculations including mate-in-N
- [ ] Unit tests cover move classification thresholds
- [ ] Unit tests cover move pair grouping logic
- [ ] Integration test: full PGN submission → analysis → results flow
- [ ] Manual test: analysis page on iPhone Safari (iOS 16+)
- [ ] Manual test: analysis page on Android Chrome (Chrome 110+)
- [ ] Manual test: keyboard navigation on desktop Chrome, Firefox, Safari

---

Appendix A — Classification Thresholds

These thresholds are based on Lichess's published classification system and are the standard in the industry.

Classification	Centipawn Loss	Color	Icon
Best Move	0 cp	Emerald (#10b981)	★
Good	1–30 cp	Green (#22c55e)	✓
Inaccuracy	31–100 cp	Amber (#f59e0b)	?!
Mistake	101–300 cp	Orange (#f97316)	?
Blunder	301+ cp	Red (#ef4444)	??

Note: "Best Move" means the player's move matches the engine's top choice exactly. "Good" means the move is within 30 centipawns of the best move — a practical equivalence for most positions.

Appendix B — Chess-API.com Integration Reference

The Chess-API.com Stockfish REST API is the engine backend for this feature. The integration is straightforward.

Request format:

```
POST https://chess-api.com/v1
Content-Type: application/json

{
  "fen": "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1",
  "depth": 12
}
```

Response fields used:

- `eval` — centipawn evaluation (positive = White advantage)
- `move` — best move in UCI notation (e.g., “e2e4”)
- `san` — best move in SAN notation (e.g., “e4”)
- `winChance` — win probability for the side to move (0.0–1.0)
- `mate` — mate-in-N value (null if no forced mate)
- `continuationArr` — array of best continuation moves

Rate limits: The free tier supports approximately 1 request per second. For a 40-move game (80 positions), analysis takes approximately 80–120 seconds at depth 12. This is acceptable for the MVP. If rate limits become a bottleneck, implement a queue with 1.2-second intervals between requests.

Error handling: The API occasionally returns 429 (rate limit) or 503 (overloaded). Implement exponential backoff with a maximum of 3 retries per position. If a position fails after 3 retries, store `eval: null` and `classification: null` for that move — do not fail the entire analysis.

Appendix C — Scope Creep Prevention

The following features have been explicitly requested or discussed but are **deferred to post-launch**. They must not be added to the MVP without a formal scope change decision.

Deferred Feature	Reason for Deferral
Video-to-board timestamp synchronization	Requires reliable vision pipeline accuracy
Opening book / ECO name display	Requires database of 3,000+ opening lines
Variation trees (alternative moves)	Significant UI complexity, not core to MVP value
Game sharing via public link	Requires access control design for shared games
Personal accuracy trends over time	Requires sufficient game history (post-launch data)
Coach annotation tools	Separate user role and permission system needed
Multi-board tournament recording	Significant infrastructure complexity
ELO performance estimation	Requires calibration against rated game data